

✧ Outras Camadas Convolucionais

Além das camadas convolucionais padrões, algumas outras surgiram tentando melhorar alguns quesitos da convolução tradicional.

Entre elas estão:

- [convolução dilatada](#) (*dilated convolution*), que permite que o filtro convolucional tenha *buracos* aumentando o *receptive field* mas mantendo a resolução da imagem, e
- [convolução separável por canal](#) (*depthwise separable convolution*), que, de acordo com a literatura, alcança resultados similares ao da convolução padrão porém usando menos parâmetros (por isso, são mais rápidas).

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F
```

```
from torch import optim
from torchsummary import summary
```

```
import time, os, sys, numpy as np
```

```
# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids= list(range(n))
```

```
def load_data_cifar10(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Cifar10-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.CIFAR10(root=root, train=True,download=True,transform=transformer)
    mnist_test = torchvision.datasets.CIFAR10(root=root, train=False,download=True,transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size, shuffle=False,
                                            num_workers=num_workers)

    return train_iter, test_iter

def load_data_fashion_mnist(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Fashion-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.FashionMNIST(root=root, train=True,download=True,transform=transformer)
    mnist_test = torchvision.datasets.FashionMNIST(root=root, train=False,download=True,transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size, shuffle=False,
                                            num_workers=num_workers)

    return train_iter, test_iter
```

```
# funções básicas
```

```

def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])

# Função usada para calcular acurácia
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""

    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]

    return acc_sum.item() / n, l.item() / len(data_iter)

# Função usada no treinamento e validação da rede
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
    print('training on', device)
    for epoch in range(num_epochs):
        net.train()
        train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
        for X, y in train_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            trainer.zero_grad()
            l = loss(y_hat, y).sum()
            l.backward()
            trainer.step()
            train_l_sum += l.item()
            train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]
        test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
        print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
              'test acc %.3f, time %.1f sec'
              % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
                 test_acc, time.time() - start))

```

✓ Convolutacional Dilatada (*Dilated Convolution*)

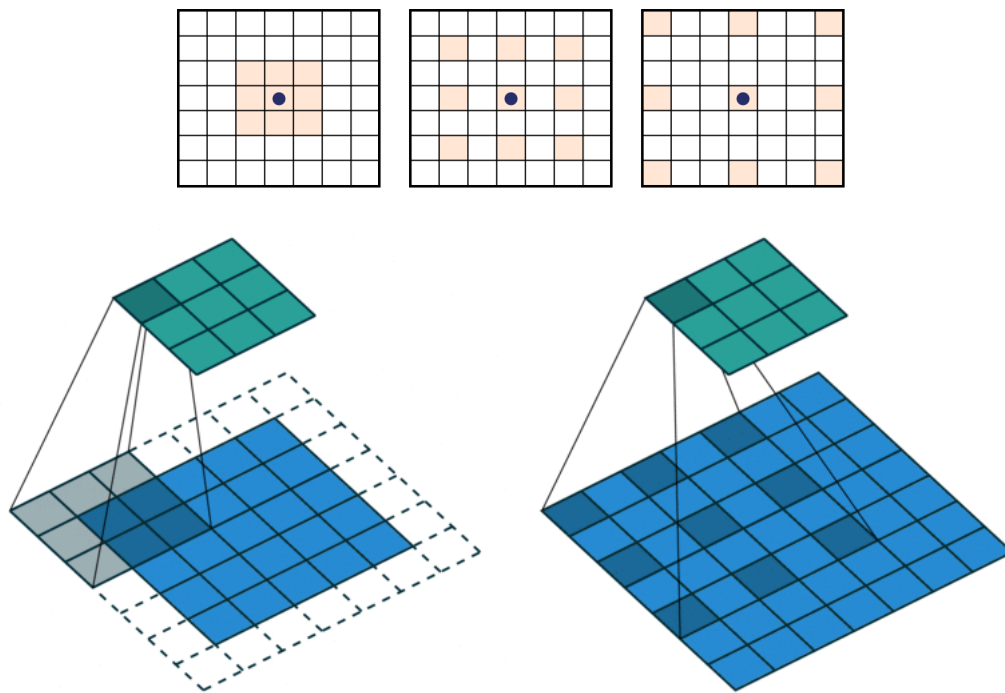
Nas camadas convolucionais dilatadas, os pesos do filtro são empregados de maneira diferente quando comparados às convoluções padrão. Especificamente, os filtros dessa camada não precisam ser contíguos e podem ter lacunas (ou "buracos") entre seus parâmetros. Essas lacunas, inseridas de acordo com a taxa de dilatação $r \in \mathbb{N}$, que permite um aumento do tamanho do filtro convolutacional, preservando o número de parâmetros treináveis, uma vez que os buracos inseridos não são considerados no processo de convolução. Portanto, esta taxa de dilatação r pode ser vista como um parâmetro responsável por definir o alinhamento final dos pesos.

Formalmente, uma convolução dilatada 2-D recebe uma entrada em duas dimensões Y , uma taxa de dilatação R e um vetor de peso 2-D W (neste caso, com tamanho $n \times n$) e processos eles dessa forma:

$$Y[k, l] = \sum_{i=1}^n \sum_{j=1}^n X(k + r \times i, l + r \times j) K(i, j)$$

, onde Y é a saída ou *feature map*. Observe as diferenças entre a dilatação e a convolução padrão apresentadas na aula anterior.

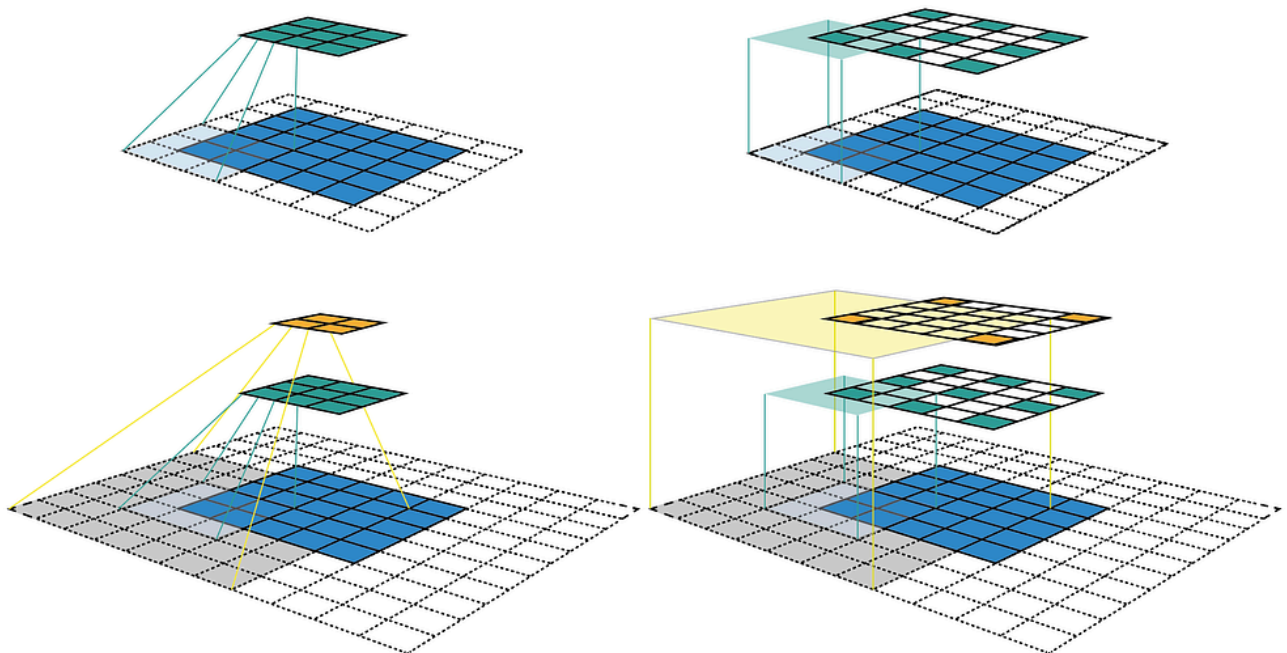
O efeito da taxa de dilatação diferente r é apresentado na figura abaixo. Como pode ser visto, taxas menores resultam em um filtro mais clusterizado (na verdade, a taxa 1 gera um filtro idêntico à convolução padrão) enquanto as taxas maiores fazem uma expansão do filtro, produzindo um kernel maior com vários buracos. Como todo esse processo de dilatação do filtro é independente dos dados de entrada, alterar a taxa de dilatação não afeta a resolução do resultado, ou seja, em uma convolução dilatada, independente da taxa, a entrada e a saída têm a mesma resolução (considerando, claro, o efeito do *padding* e do *stride*).



Ao ampliar o filtro (com essas lacunas), a rede expande seu campo receptivo (já que os pesos serão organizados em uma forma mais esparsa), mas preserva a resolução e nenhuma redução de resolução nos dados é executada. Portanto, esse processo tem várias vantagens, como: (i) suporta a expansão do campo receptivo sem aumentar o número de parâmetros treináveis por camada, o que reduz a carga computacional, e (ii) preserva a resolução do mapa de características, o que pode ajudar a rede a extrair informações ainda mais úteis dos dados, principalmente de pequenos objetos.

Campo Receptivo (*Receptive Field*)

O campo receptivo é definido como a região no espaço de entrada que tem influência sobre a saída atual da rede convolucional.



Implementação

No Pytorch, a [camada convolução padrão](#) tem suporte para dilatação dos filtros de acordo com o parâmetro *dilation*.

```
class LeNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):
        super(LeNet, self).__init__(**kwargs)

        self.conv1 = nn.Conv2d(in_channels=input_channels, out_channels=6, kernel_size=5, stride=1, padding=0)
        self.avgpool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0, dilation=3)
        # self.avgpool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        # self.conv3 = nn.Conv2d(in_channels=16, out_channels=120, kernel_size=5, stride=1, padding=0)
        self.flatten = nn.Flatten()
```

```

        self.fc1 = nn.Linear(64, 84)
        self.fc2 = nn.Linear(84, classes)

    def forward(self, x):

        output_conv1 = F.tanh(self.conv1(x))
        output_avgpool1 = self.avgpool1(output_conv1)
        output_conv2 = F.tanh(self.conv2(output_avgpool1))
        # output_avgpool2 = self.avgpool2(output_conv2)
        # output_conv3 = F.tanh(self.conv3(output_avgpool2))
        flatten_output = self.flatten(output_conv2)
        output_fc1 = F.tanh(self.fc1(flatten_output))
        output = self.fc2(output_fc1)

    return output

```

```

# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 10, 0.1, 128, 0.000001

# rede baseada na AlexNet-5
net = LeNet(1, 10)

# Sending model to device
net.to(device)
print(summary(net,(1,32,32))) # visualize number of parameters' net, output of each layer and total mega bytes necessary for
# and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do gluon
trainer = optim.SGD(net.parameters(), lr=lr, weight_decay=wd_lambda, momentum=0.9)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)

```

```

-----
Layer (type)          Output Shape          Param #
-----
Conv2d-1              [-1, 6, 28, 28]       156
AvgPool2d-2           [-1, 6, 14, 14]       0
Conv2d-3              [-1, 16, 2, 2]        2,416
Flatten-4              [-1, 64]              0
Linear-5               [-1, 84]              5,460
Linear-6               [-1, 10]              850
-----

```

```

Total params: 8,882
Trainable params: 8,882
Non-trainable params: 0

```

```

-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.05
Params size (MB): 0.03
Estimated Total Size (MB): 0.08
-----

```

```

None
training on cuda
/usr/local/lib/python3.6/dist-packages/torch/nn/functional.py:1628: UserWarning: nn.functional.tanh is deprecated. Use torch
warnings.warn("nn.functional.tanh is deprecated. Use torch.tanh instead.")
epoch 1, train loss 0.5977, train acc 0.780, test loss 0.4687, test acc 0.826, time 8.7 sec
epoch 2, train loss 0.4035, train acc 0.854, test loss 0.4263, test acc 0.843, time 8.7 sec
epoch 3, train loss 0.3740, train acc 0.864, test loss 0.4107, test acc 0.851, time 9.2 sec
epoch 4, train loss 0.3496, train acc 0.872, test loss 0.3977, test acc 0.853, time 8.9 sec
epoch 5, train loss 0.3364, train acc 0.875, test loss 0.3938, test acc 0.859, time 8.4 sec
epoch 6, train loss 0.3338, train acc 0.877, test loss 0.3652, test acc 0.868, time 8.8 sec
epoch 7, train loss 0.3225, train acc 0.879, test loss 0.3815, test acc 0.856, time 8.9 sec
epoch 8, train loss 0.3176, train acc 0.881, test loss 0.3932, test acc 0.851, time 8.8 sec
epoch 9, train loss 0.3090, train acc 0.884, test loss 0.3575, test acc 0.874, time 9.4 sec
epoch 10, train loss 0.3050, train acc 0.885, test loss 0.3568, test acc 0.871, time 9.0 sec

```

Para efeito de **comparação**, recriamos a rede sem as duas camadas convolucionais (removidas na arquitetura anterior) e sem camadas dilatadas. Dessa forma, podemos observar o ganho das convoluções dilatadas.

```

class LeNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):

```

```

super(LeNet, self).__init__(**kwargs)

self.conv1 = nn.Conv2d(in_channels=input_channels,out_channels=6, kernel_size=5, stride=1, padding=0)
self.avgpool1 = nn.AvgPool2d(kernel_size=2, stride=2)
self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0)
self.avgpool2 = nn.AvgPool2d(kernel_size=2, stride=2)
self.conv3 = nn.Conv2d(in_channels=16, out_channels=120, kernel_size=5, stride=1, padding=0)
self.flatten = nn.Flatten()
self.fc1 = nn.Linear(120, 84)
self.fc2 = nn.Linear(84, classes)

def forward(self, x):

    output_conv1 = F.tanh(self.conv1(x))
    output_avgpool1 = self.avgpool1(output_conv1)
    output_conv2 = F.tanh(self.conv2(output_avgpool1))
    output_avgpool2 = self.avgpool2(output_conv2)
    output_conv3 = F.tanh(self.conv3(output_avgpool2))
    flatten_output = self.flatten(output_conv3)
    output_fc1 = F.tanh(self.fc1(flatten_output))
    output = self.fc2(output_fc1)

    return output

```

```

# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 10, 0.1, 128, 0.000001

# rede baseada na AlexNet-5
net = LeNet(1, 10)

# Sending model to device
net.to(device)
print(summary(net,(1,32,32))) # visualize number of parameters' net, output of each layer and total mega bytes necessary for
# and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do gluon
trainer = optim.SGD(net.parameters(), lr=lr, weight_decay=wd_lambda, momentum=0.9)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)

```

```

-----
Layer (type)          Output Shape          Param #
-----
Conv2d-1              [-1, 6, 28, 28]       156
AvgPool2d-2           [-1, 6, 14, 14]       0
Conv2d-3              [-1, 16, 10, 10]      2,416
AvgPool2d-4           [-1, 16, 5, 5]        0
Conv2d-5              [-1, 120, 1, 1]       48,120
Flatten-6             [-1, 120]             0
Linear-7              [-1, 84]              10,164
Linear-8              [-1, 10]              850
=====
Total params: 61,706
Trainable params: 61,706
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.06
Params size (MB): 0.24
Estimated Total Size (MB): 0.30
-----
None
training on cuda
/usr/local/lib/python3.6/dist-packages/torch/nn/functional.py:1628: UserWarning: nn.functional.tanh is deprecated. Use torch
warnings.warn("nn.functional.tanh is deprecated. Use torch.tanh instead.")
epoch 1, train loss 0.6267, train acc 0.763, test loss 0.4404, test acc 0.843, time 8.7 sec
epoch 2, train loss 0.3904, train acc 0.855, test loss 0.3763, test acc 0.862, time 8.9 sec
epoch 3, train loss 0.3527, train acc 0.869, test loss 0.3716, test acc 0.859, time 8.8 sec
epoch 4, train loss 0.3239, train acc 0.881, test loss 0.3807, test acc 0.857, time 8.9 sec
epoch 5, train loss 0.3029, train acc 0.887, test loss 0.3615, test acc 0.869, time 9.1 sec
epoch 6, train loss 0.2868, train acc 0.894, test loss 0.3199, test acc 0.886, time 8.7 sec
epoch 7, train loss 0.2670, train acc 0.902, test loss 0.3185, test acc 0.882, time 8.8 sec
epoch 8, train loss 0.2594, train acc 0.904, test loss 0.3179, test acc 0.890, time 9.2 sec

```

```
epoch 9, train loss 0.2470, train acc 0.907, test loss 0.3216, test acc 0.886, time 9.0 sec
epoch 10, train loss 0.2392, train acc 0.911, test loss 0.3300, test acc 0.885, time 9.0 sec
```

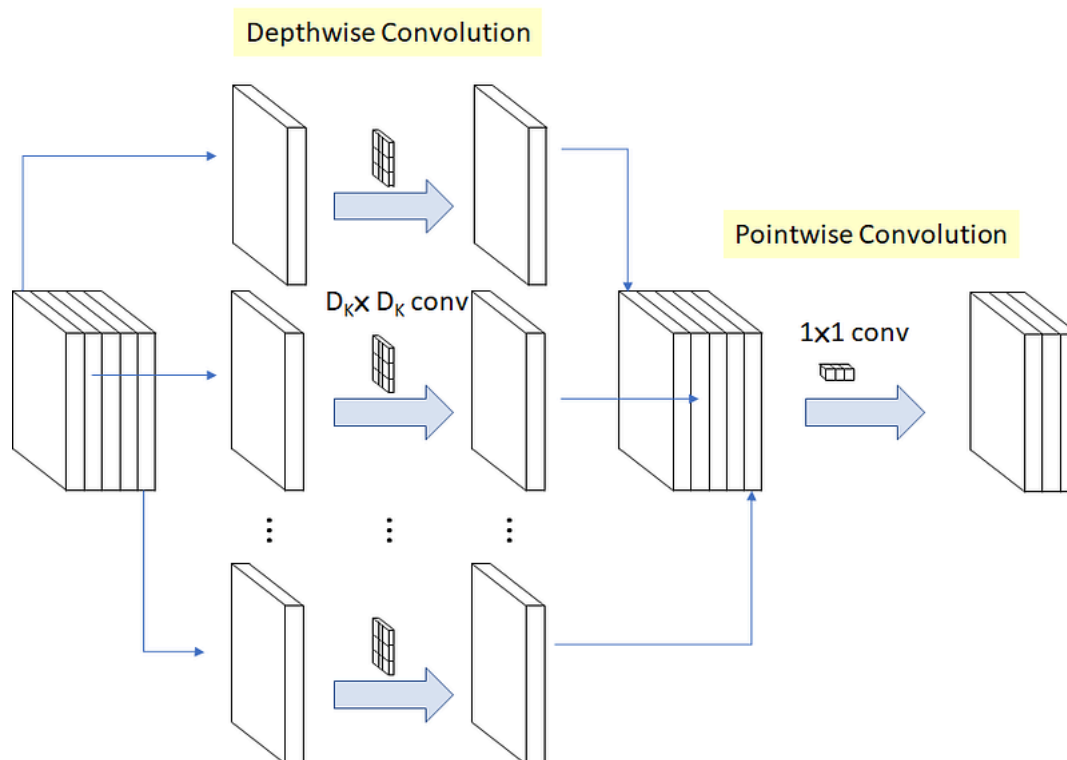
A acurácia atingida no teste é praticamente a mesma para os dois modelos. No entanto, na rede com dilation o número de parâmetros é quase 4 vezes menor, sendo assim, uma probabilidade menor de acontecer um overfitt.

✓ Convolução Separável por Canal (*Depthwise Separable Convolution*)

Após o resurgimento das redes neurais, a cada nova arquitetura proposta, mais e mais camadas e parâmetros foram sendo usados. Por exemplo, a primeira arquitetura dessa nova onda, a [AlexNet](#), tem **61.100.840** de parâmetros.

Claramente, pelo tamanho de parâmetros, arquitetura como essas não são passíveis de serem treinadas em dispositivos com menos memória, como sistema embarcados, celulares, tablets, etc. Por isso, uma linha de pesquisa procura otimizar o uso dos parâmetros em rede neurais. Para esse fim, essa linha de pesquisa propôs uma nova camada, chamada Convolução Separável por Canal (*Depthwise Separable Convolution*), que é capaz de reproduzir arquiteturas convolucionais, obtendo resultados similar, porém com menos parâmetros.

A ideia por trás dessa camada é representada abaixo. Ao invés de ter uma única camada de convolução que processará toda a entrada (ou seja, terá parâmetros para todos os canais de entrada), essa convolução é dividida em duas partes. Na primeira parte, cada canal é processado separadamente por um filtro específico. Formalmente, suponha que a entrada tenha c_i canais. Nesse caso, a primeira parte de convolução terá c_i filtros de tamanho $k \times k$, onde cada um desses filtros processará somente um canal da entrada, gerando c_i *feature maps* de saída. Na segunda parte, esses *feature maps* serão processados por uma convolução 1×1 , que gerará a saída final esperada. Suponha que queiramos que essa camada gere c_o canais de saída. Essa segunda parte terá c_o filtros de 1×1 que processarão os *feature maps* gerados na primeira parte, gerando a saída final com c_o canais.



Vamos examinar a diferença entre a convolução padrão e a convolução separável por canal em termos de número de parâmetros. Suponha que:

- n , seja a dimensão espacial (largura e altura), da entrada,
- k , seja largura e altura do filtro,
- c_i , seja o número de canais de entrada, e
- c_o , o número de canais de saída.

Uma convolução regular tem $k * k * c_i * c_o$ parâmetros, porque, para cada canal de saída, há um filtro $k \times k$ para cada canal de entrada. Em contrapartida, as convoluções separáveis por canal tem $k * k * c_i$ parâmetros na primeira parte e, em seguida, $1 * 1 * c_i * c_o$ parâmetros para a segunda parte. Deve ser óbvio que, para um c_o não trivial, a soma desses dois é significativamente menor que $k * k * c_i * c_o$.

Agora, vamos comparar o custo computacional. Para uma convolução regular, realizamos $K * K * c_i$ operações em cada posição da entrada (para calcular a convolução 2D em 3 dimensões). Para toda a entrada, o número de cálculos é, portanto, $k * k * c_i * n * n$ e tomando todos os canais de saída, obtemos $k * k * c_i * n * n * c_o$. Para convoluções separáveis em profundidade, precisamos de operações $k * k * c_i * n * n$ para a parte de profundidade; então precisamos de $n * n * c_i * c_o$ operações para a parte de mistura.

Vamos usar alguns números reais para sentir a diferença. Assumiremos

- $n = 128$,
- $k = 3$,
- $c_i = 3$,
- $c_o = 16$.

Para convolução regular:

Parâmetros: $3 * 3 * 3 * 16 = 432$

Custo de computação: $3 * 3 * 3 * 128 * 128 * 16 = \sim 7e6$

Para a convolução separável em profundidade:

Parâmetros: $3 * 3 * 3 + 3 * 16 = 75$

Custo de computação: $3 * 3 * 3 * 128 * 128 + 128 * 128 * 3 * 16 = \sim 1.2e6$

Frameworks modernos implementam esse tipo de camada de forma diferente. No Pytorch, nosso caso de estudo, a primeira parte dessa convolução é feita quando se coloca o [número de grupos da camada convolucional igual ao número de canais de entrada](#). Esse [parâmetro](#) é usado para definir o agrupamento entre canais de entrada e filtros/neurônios. Logo, quando se tem um número de canais de entrada igual ao número de filtros, e também igual ao número de grupos. Dessa forma, cada filtro processará somente um canal.

Já a segunda parte dessa convolução é feita naturalmente, usando o *kernel*=1.

Abaixo, implementamos a LeNet-5 usando essa convolução.

	Layer	Feature Map	Size	Kernel Size	Stride	Activation
	Input	Image	1	32x32	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

```
class LeNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):
        super(LeNet, self).__init__(**kwargs)
        self.conv1_1 = nn.Conv2d(in_channels=input_channels, out_channels=1, kernel_size=5, stride=1, padding=0, groups=1)
        self.conv1_2 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=1, stride=1, padding=0)
        self.avgpool1 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv2_1 = nn.Conv2d(in_channels=6, out_channels=6, kernel_size=5, stride=1, padding=0, groups=6)
        self.conv2_2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=1, stride=1, padding=0)
        self.avgpool2 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv3_1 = nn.Conv2d(in_channels=16, out_channels=16, kernel_size=5, stride=1, padding=0, groups=16)
        self.conv3_2 = nn.Conv2d(in_channels=16, out_channels=120, kernel_size=1, stride=1, padding=0)

        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(120, 84)
        self.fc2 = nn.Linear(84, classes)

    def forward(self, x):
        # print('x', x.shape)
        output_conv1_1 = self.conv1_1(x)
        # print('conv1_1', x.shape)
        output_conv1_2 = F.tanh(self.conv1_2(output_conv1_1))
        # print('conv1_2', x.shape)
        output_avgpool1 = self.avgpool1(output_conv1_2)
        output_conv2_1 = self.conv2_1(output_avgpool1)
        # print('conv2_1', x.shape)
        output_conv2_2 = F.tanh(self.conv2_2(output_conv2_1))
        # print('conv2_2', x.shape)
        output_avgpool2 = self.avgpool2(output_conv2_2)
        output_conv3_1 = self.conv3_1(output_avgpool2)
```

```
# print('conv3_1', x.shape)
output_conv3_2 = F.tanh(self.conv3_2(output_conv3_1))
# print('conv3_2', x.shape)
output_flatten = self.flatten(output_conv3_2)
output_fc1 = F.tanh(self.fc1(output_flatten))
output = self.fc2(output_fc1)

return output
```

```
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 10, 0.01, 128, 0.000001

# rede
net = LeNet(1, 10)

# Sending model to device
net.to(device)
print(summary(net,(1,32,32))) # visualize number of parameters' net, output of each layer and total mega bytes necessary for
# and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do gluon
trainer = optim.Adam(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)
```

```
-----
Layer (type)                 Output Shape              Param #
=====
Conv2d-1                     [-1, 1, 28, 28]           26
Conv2d-2                     [-1, 6, 28, 28]           12
AvgPool2d-3                  [-1, 6, 14, 14]           0
Conv2d-4                     [-1, 6, 10, 10]           156
Conv2d-5                     [-1, 16, 10, 10]          112
AvgPool2d-6                  [-1, 16, 5, 5]            0
Conv2d-7                     [-1, 16, 1, 1]            416
Conv2d-8                     [-1, 120, 1, 1]           2,040
Flatten-9                    [-1, 120]                  0
Linear-10                    [-1, 84]                   10,164
Linear-11                    [-1, 10]                    850
=====
Total params: 13,776
Trainable params: 13,776
Non-trainable params: 0
```